Chair of IT Security
Brandenburg University of Technology

Dr.-Ing. Asya Mitseva
Prof. Dr.-Ing. Andriy Panchenko

Winter Semester 2026/2027

Part II: Implementation of Statistical Detection Methods **Deadline: 18.11.2026**

Study Project: Adversarial Machine Learning
Generation of Datasets for Anomaly Detection in Industrial Control Systems

Introduction

Industrial Control Systems (ICS) monitor and control critical infrastructures such as chemical plants, power systems, gas pipelines, and water systems. Typically, ICS consist of two macro areas, known as Information Technology (IT) and Operational Technology (OT). While IT is composed of standard computers interconnected via traditional communication protocols, OT includes special hardware and software for monitoring and controlling a given industrial facility. Overall, OT consists of *field*, *control*, and *supervisory* layers. The field layer includes different sensors and actuators, while the control layer is composed of a number of programmable logic controllers (PLCs). The sensors measure the current state of the underlying physical process, which is then reported to the PLCs. The PLCs implement a control logic used to evaluate the obtained sensor readings according to predefined or dynamically adjustable target values. Based on this evaluation, the PLCs may initiate specific commands to one or more actuators in the field layer aiming to adjust the current state of the physical process. In the supervisory layer, a Supervisory Control and Data Acquisition (SCADA) system performs a high-level visualization and control over the control and field layers and provides an overview of the current process state to the operator. While OT was mainly a standalone system in the past, nowadays it incorporates both decades-old devices and communication infrastructure, and new generation embedded devices with computing capabilities and Ethernet-based communication protocols, and is more often connected to the Internet. While the digitization of ICS makes the monitoring and the control of the physical process easier, it creates new attack surfaces against ICS.

A major line of research focused on the design and the development of robust intrusion detection systems (IDSs) for the identification of cyber attacks in ICS. As changes in OT usually follow immutable laws of physics, many IDSs rely on a bunch of sensor values and actuator states collected from the OT to detect attacks. However, the main challenge here is the lack of representative datasets needed for these IDSs. A straightforward solution would be to collect a bunch of physical readings from a real ICS. However, ICS operators are often not willing to share such data due to confidentiality reasons. The collection of data containing cyber attacks from a real ICS (i.e., executing attacks there) is even more challenging. Therefore, the goal of this study project is to implement and analyze different approaches for generating synthetic, but at the same time realistic datasets.

In the first practical sheet, we got familiar with a popular ICS dataset, namely WADI, and implemented our own approach for the generation of synthetic dataset. In the next part of the study project, we analyze the suitability of a newly generated synthetic data for anomaly detection and start implementing different statistical anomaly detection methods to detect possible anomalies / attacks in ICS.

Task 1: Statistical Analysis of Synthetic Dataset

For this task, you should only use both sets of data comprising normal behavior from both

datasets (WADI 2017 and WADI 2019) that you used in the previous practical sheet. Next, you should use the implementation of your over-sampling method from Task 3 in the previous practical sheet and one day of normal data from exactly one of these datasets to train your over-sampling method and generate synthetic data for 2.5 additional days for each of the different $k = \{3, 5, 7\}$. The obtained synthetic data should be also submitted in Moodle. Finally, you should compute the following statistics:

a) From the set of normal data from the same dataset that was not used for training the over-sampling method, you should create two subsets of data comprising 2.5 days. Then, all new sets of data (i.e., both original ones and the synthetically generated ones) should be normalized.

b) You should analyze the similarity of the different datasets, the synthetically generated ones and the normal data set from Item a). To this end, you should compute different statistical methods used to identify how much they differentiate between each other. Each of the statistical methods below should be implemented to learn a single model per a communication pair and protocol from a given dataset. In particular, the statistical methods should be as follows:

- *MinMax:* This statistic should compute the minimum and maximum values of a given sensor and/or actuator from the real dataset and identify *whether* and *how much* a current value for the same sensor and/or actuator observed in each of the synthetic datasets exceeds this range. To compute the predefined range, you should introduce an error margin to the learned thresholds as follows: $min_{err} = min - \frac{max-min}{4}$ and $max_{err} = max + \frac{max-min}{4}$.
- *Gradient:* This statistic should identify *whether* and *how much* a sensor's and/or actuator's slope in a given synthetic dataset exceeds the minimum and maximum slope observed in the real dataset. The Gradient approach assumes that physical readings have continual character, i.e., physical values such as temperatures cannot change at arbitrary speed. To compute the predefined range, you should again introduce an error margin to the learned thresholds as follows: $min_{err} = min - \frac{max-min}{4}$ and $max_{err} = max + \frac{max-min}{4}$. You are not allowed to use any preexisting function from a library for the computation of the gradient.
- *Steadytime:* This statistic should identify *whether* and *how much* a sensor and/or actuator remains static in a given synthetic dataset, i.e., does not change its value, for a shorter or longer time than the one seen in the real dataset. Since a steady state is difficult to define for noisy sensor data, Steadytime should take only process values into account if the number of distinct values during training is sufficiently small ($<= 50$). To compute the predefined range, you should again introduce an error margin to the learned thresholds as follows: $min_{err} = min - \frac{max-min}{4}$ and $max_{err} = max + \frac{max-min}{4}$.
- *Entropy:* For each dataset (both the real and the synthetic ones), this statistic should identify *whether* the values for a given sensor and/or actuator repeat often or they are rather unique. For the computation of entropy per sensor/actuator, you should apply *Shannon entropy*. Please note that for the computation of Shannon entropy you are not allowed to use any predefined library.

After computing and storing all statistics using each of the statistical methods above for the datasets, you should write a piece of code that plots (i) a cumulative distribution function (CDF) for the observed entropy values in each of the datasets and (ii) a separate complementary cumulative distribution function (CCDF) for the MinMax statistic showing *whether* and *how much* the synthetically generated packet sizes differentiate from the corresponding values in the real dataset. Beside the source code, you should submit all generated plots for this task.

Task 2: CUSUM and Exponential Weighted Moving Average (EWMA)

The goal of this task is to implement CUmulative SUM (CUSUM) and exponential weighted moving average (EWMA) statistics in order to detect attacks using the dataset that you generated in the previous task. In particular, your piece of code should satisfy the following requirements:

a) Implement a non-parametric CUSUM statistic that is defined recursively as $S_0 = 0$ and $S_{k+1} = (S_k + |r_k| - \beta)^+$, where r_k is the difference between a previous and current measurement, $(S_k + |r_k| - \beta)^+$ represents $max(0, (S_k + |r_k| - \beta))$, and β is selected during the training phase so that the expected value of $|r_k| - \beta < 0$ under hypothesis H_0, i.e., β prevents S_k from increasing consistently under normal operation. An alert is generated whenever the statistic is greater than a previously defined threshold $S_k > \theta$ and the test is restarted with $S_{k+1} = 0$.

b) Implement an EWMA statistic that is defined recursively as $S_0 = y_k$ and $S_{k+1} = (\alpha \cdot y_{k+1} + (1 - \alpha) \cdot S_k)$, where y_k is a sensor measurement at time period t and α represents the degree of weighting decrease, which should be fine-tuned during the training phase. An alert is generated whenever the statistic is greater than a previously defined threshold $S_k > \theta$ and the test is restarted with $S_{k+1} = 0$.

c) For each of both methods, you should use the intermediate alarms from single sensors/actuators to develop your own method that is able to take final decision whether there is an attack or not in a given time step.

d) The user should be able to select from the command line which of both statistical algorithms to use.

Task 3: Automatic Generation of Features

The goal of this task is to implement a deep neural network (DNN) that performs automated, unsupervised extraction of features from the physical readings of all sensors and actuators. In particular, you should develop a variational autoencoder (VAE) that takes as an input m x n sets of physical readings, where m indicates the number of the sets of physical readings and n the set of physical readings in a single set of physical readings. In addition, your piece of code should satisfy the following requirements:

a) Your VAE should take as an input predefined sets of physical readings whereas each of these sets should consist of $n_p = M$ physical readings and M should be a parameter that is read from the command line.

b) Your VAE model should support *at least* three different types of layers and *at least* three different activation functions (i.e., you should create different VAE models for the different layers and the different activation functions). In addition, your piece of code should be able to print out the features obtained after training for each of the different VAE models in a file and it should be clear which set of features belongs to which set of physical readings.

c) For each VAE model, you should provide an implementation for the search of optimal hyperparameters.

d) Each VAE model should contain *at least* three hidden layers in the encoder part and the number of features that are obtained after encoding should be a parameter that is read from the command line.

e) Each VAE model should contain two decoder parts and the user should be able to choose which one to use. The first decoder should be able to reconstruct the initial sets of physical readings and the *Kullback-Leiber divergence (KLD)* should be used as a reconstruction error. Instead of reconstructing the initial sets of physical readings, the second decoder should act as a classification branch and the *categorical cross-entropy* should be optimized during VAE training.

Preparation for defense

Prepare yourself for a defense of approximately 90 minutes. During the defense, you need to execute implementation changes to multiple parts of the practical sheet using your source code that you submitted in Moodle. The changes implemented during the defense should be submitted and graded at the end of the session. During the defense, you will use computers provided by us that run Linux Ubuntu 22.04.5 LTS. You can only use Geany, Nano, Vim, or Neovim on these computers to implement changes in the source code, as well as the terminal to test and run your modified source code. To set up the Python environment, you can use Anaconda or venv. Please note that you will not have administrative rights to install additional software. If you indeed need additional software for your practical sheet, you must announce this via email at least 2 working days in advance. Missing software cannot be used as an excuse to not complete or attend the defense.